

PHYS 410 - Homework 2

Felipe Garavelli
46427951

November 3, 2025

Contents

1	Introduction	2
2	Theory	2
2.1	Runge-Kutta 4th Order Method	2
2.2	Adaptive Step-Size Control	2
3	Problem 1: Single RK4 Step	3
3.1	Numerical Approach and Implementation	3
3.2	Results	4
4	Problem 2: Fixed-Step Integrator	4
4.1	Numerical Approach and Implementation	4
4.1.1	Simple Harmonic Oscillator (SHO): Global Error Verification	5
4.1.2	Van der Pol Oscillator (VDP): Non-linear Application	5
4.2	Results	5
4.2.1	SHO	5
4.2.2	VDP	5
5	Problem 3: Adaptive RK4 Integrator	7
5.1	Numerical Approach and Implementation	7
5.2	Results	8
5.2.1	SHO	8
5.2.2	VDP	8
6	Conclusion	11
6.1	Use of AI	11
7	Files	11

1 Introduction

This report details the implementation and verification of numerical integrators based on the classic fourth-order Runge-Kutta (RK4) method for solving systems of ordinary differential equations. The work progresses from a single-step function (**rk4step**) to a fixed-step integrator (**rk4**) and finally to an adaptive-step solver (**rk4ad**). These integrators were tested against two well-defined problems: the Simple Harmonic Oscillator (SHO), which has a known analytical solution for precise error checking, and the unforced Van der Pol oscillator (VDP), which exhibits a non-linear limit cycle.

2 Theory

2.1 Runge-Kutta 4th Order Method

To solve an initial value problem of the form $y'(t) = f(t, y)$ with $y(t_0) = y_0$, the fourth-order Runge-Kutta method advances the solution from t_n to $t_{n+1} = t_n + h$ using a weighted average of four slope estimates.

The formula for the “classic” RK4 method is:

$$y_{n+1} = y_n + \frac{h}{6}(f_1 + 2f_2 + 2f_3 + f_4) \quad (1)$$

where h is the step size and the f_i terms are:

$$\begin{aligned} f_1 &= f(t_n, y_n) \\ f_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f_1\right) \\ f_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f_2\right) \\ f_4 &= f(t_n + h, y_n + hf_3) \end{aligned}$$

The RK4 method is a single-step, explicit method. Its local truncation error (the error incurred in a single step) is $O(h^5)$, and its total accumulated (global) error is $O(h^4)$.

2.2 Adaptive Step-Size Control

To achieve a desired accuracy without using an excessively small step size, an adaptive integrator can estimate the local error and adjust the step size h (or Δt). The method used here compares the result of one “coarse” step of size Δt with two “fine” steps of size $\Delta t/2$.

Let y_C be the result of the coarse step and y_F be the result of the two fine steps. The local error for the coarse step, $e_C = y_C - y_{\text{exact}}$, can be shown to be $e_C \approx k\Delta t^5$ for some constant k .

The local error for the two fine steps, e_F , is the sum of the errors from each step. The error for the two fine steps is:

$$e_F = k(\Delta t/2)^5 + k(\Delta t/2)^5 + O(\Delta t) \approx 2k \left(\frac{\Delta t^5}{32} \right) = \frac{k\Delta t^5}{16} = \frac{e_C}{16} \quad (2)$$

The computable difference, ϵ , between the two estimates is:

$$\epsilon = y_C - y_F = (y_{\text{exact}} + e_C) - (y_{\text{exact}} + e_F) = e_C - e_F$$

Substituting the expression for e_F from equation (2) gives:

$$\epsilon \approx e_C - \frac{e_C}{16} = \frac{15}{16}e_C$$

This gives a useful result: we can estimate the (unknown) local error e_C using the (known) difference ϵ :

$$e_C \approx \frac{16}{15}\epsilon = \frac{16}{15}\|y_C - y_F\|$$

This error estimate, e_C , can be compared to a desired tolerance, tol . The optimal new step size, Δt_{new} , can be found using the $O(\Delta t^5)$ relationship:

$$\frac{\Delta t_{\text{new}}}{\Delta t_{\text{old}}} \approx \left(\frac{\text{tol}}{e_C} \right)^{1/5}.$$

3 Problem 1: Single RK4 Step

3.1 Numerical Approach and Implementation

The `rk4step.m` function was implemented with the header `function yout = rk4step(fcn, t0, dt, y0)`. It calculates f_1, f_2, f_3, f_4 and y_{out} precisely as defined in Section 2.1.

Once the step function was implemented, the primary part of this problem was to design a test to verify the function's local (per step) error. The theory predicts this error, E , should be proportional to the fifth power of the step size, $E \propto \Delta t^5$. To test this, a script `trk4step.m` was developed to:

1. Define a test problem with a known analytical solution. I chose a component of the Simple Harmonic Oscillator, $\frac{d^2x}{dt^2} = -x$, which converts to the first-order system:

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= -y_1 \end{aligned}$$

Given the initial condition $y(0) = [1; 0]$, the exact solution is $y(t) = [\cos(t); -\sin(t)]$.

2. Perform a *single* step from $t = 0$ using a range of step sizes Δt .
3. Compute the 2-norm of the true local error, $E = \|y_{\text{true}}(\Delta t) - y_{\text{numeric}}\|_2$, for each Δt .

If the implementation and theory are correct, a log-log plot of E versus Δt should produce a straight line with a slope of 5. This is because $E = C \cdot \Delta t^5$ becomes $\log(E) = \log(C) + 5\log(\Delta t)$, which is the equation of a line.

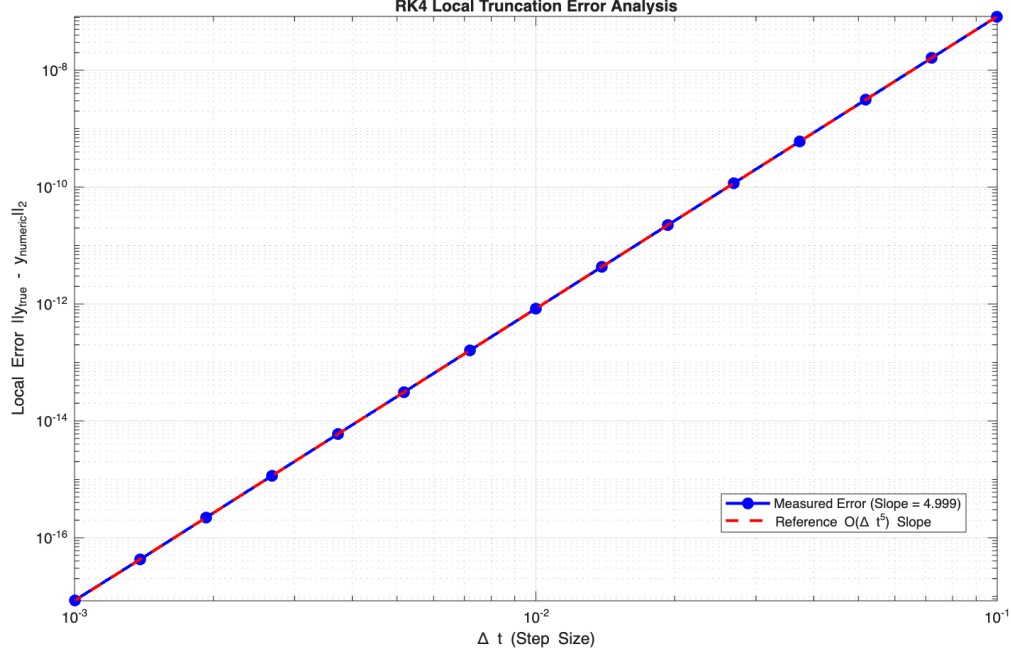


Figure 1: Log-log plot of local error vs. step size (Δt) for `rk4step`. The measured slope in the convergence region is compared to a reference line with slope 5.

3.2 Results

The results of the local error analysis are shown in Figure 1.

As seen in the figure, the measured error (blue dots) follows the reference $O(\Delta t^5)$ line (red dashed) almost perfectly for step sizes between 10^{-3} and 10^{-1} . A linear fit to this data yields a slope of approximately 5.00 (as shown in the figure), numerically confirming the theoretical $O(\Delta t^5)$ local truncation error of the RK4 method.

4 Problem 2: Fixed-Step Integrator

4.1 Numerical Approach and Implementation

The `rk4.m` function was implemented with the header function `[tout, yout] = rk4(fcn, tspan, y0)`. This function serves as a “wrapper” that integrates an ODE system over a specified time span by repeatedly calling the `rk4step` function from Section 3. The `rk4.m` function initializes an output array, `yout`, to store the solution at each time. It then iterates through the `tspan` vector, computing the step size Δt as the difference between adjacent time points. In each loop, it calls `rk4step` to advance the solution from the current time to the next, storing the result in the corresponding row of `yout`.

This fixed-step integrator was tested and verified using two distinct systems, as described in the script `trk4_sho.m` and `trk4_vdp.m`.

4.1.1 Simple Harmonic Oscillator (SHO): Global Error Verification

To verify the integrator's global error rate, the system

$$\frac{d^2x}{dt^2} = -x,$$

which in first-order form is

$$\begin{aligned}y_1' &= y_2 \\ y_2' &= -y_1,\end{aligned}$$

was solved with $y(0) = [0; 1]$ on $t \in [0, 3\pi]$. The exact solution is $y(t) = [\sin(t); \cos(t)]$. The numerical approach was to solve the problem at three resolutions: level $\ell = 6$ (2^6 steps), $\ell = 7$, and $\ell = 8$. Since the global error E is $O(\Delta t^4)$, halving Δt should reduce the error by a factor of $2^4 = 16$. We therefore expect $E_6 \approx 16 \times E_7 \approx 256 \times E_8$. The script plots these three scaled error curves to check for overlap, which would confirm the $O(\Delta t^4)$ convergence.

4.1.2 Van der Pol Oscillator (VDP): Non-linear Application

To test the integrator on a non-linear problem, the unforced Van der Pol system,

$$\frac{d^2x}{dt^2} - a(1 - x^2)\frac{dx}{dt} + x = 0,$$

which in first-order form is

$$\begin{aligned}y_1' &= y_2 \\ y_2' &= a(1 - y_1^2)y_2 - y_1,\end{aligned}$$

was solved with $a = 5$ on $t \in [0, 100]$ at level 12. The initial conditions were $y(0) = [1; -6]$. The numerical approach for this problem was to generate two plots to visualize the system's behavior: (1) the oscillator's position $x(t)$ versus time, and (2) the phase-space evolution dx/dt versus $x(t)$, which is expected to show the solution converging to a limit cycle.

4.2 Results

4.2.1 SHO

The scaled error plot (Figure 2) shows the three curves for E_6 , $16 \times E_7$, and $256 \times E_8$ lying almost perfectly on top of one another. This provides strong numerical evidence that the `rk4` integrator has a global convergence rate of $O(\Delta t^4)$, as expected from the theory.

4.2.2 VDP

The simulation of the Van der Pol oscillator was successful. The position-time plot (Figure 3) shows the solution quickly settling into a stable, periodic oscillation after an initial transient. The phase-space plot (Figure 4) clearly shows the trajectory spiraling into the known limit cycle for this system after that initial oscillation.

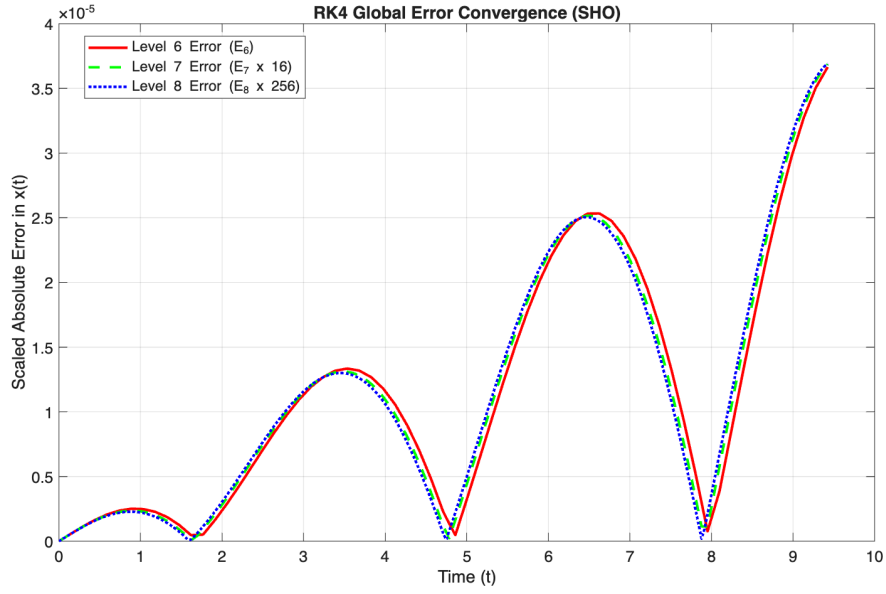


Figure 2: Scaled global error for the SHO solved with `rk4` at levels 6, 7, and 8. The overlap of the curves confirms $O(\Delta t^4)$ convergence.

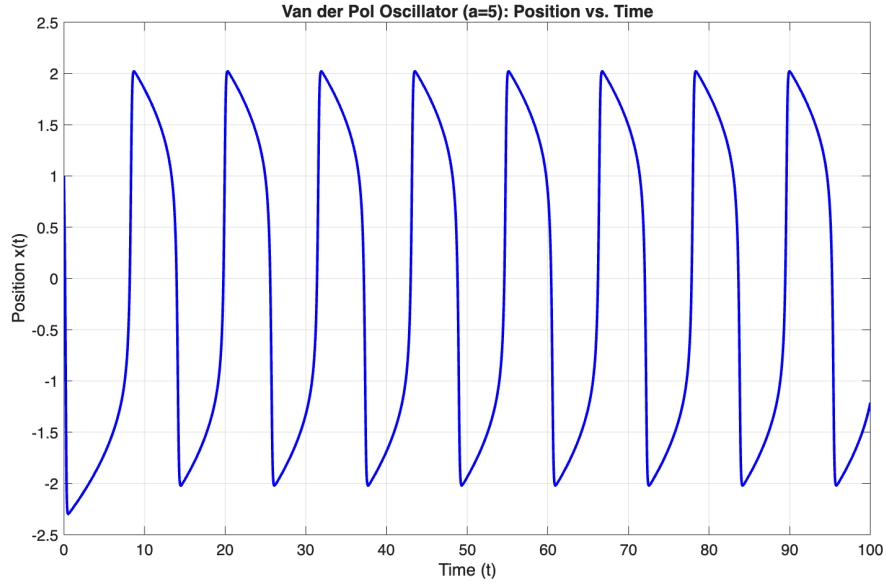


Figure 3: Position vs. Time for the Van der Pol oscillator ($a=5$) using `rk4`.

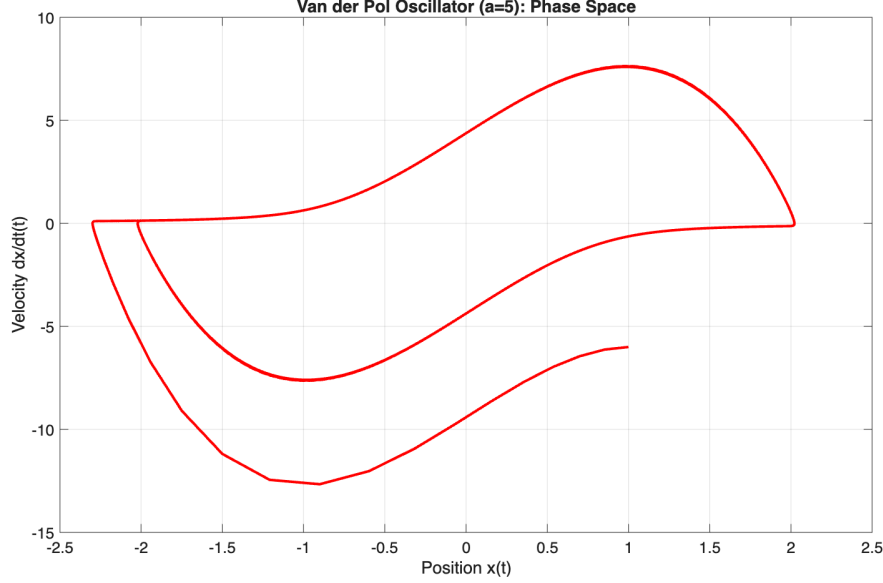


Figure 4: Phase-space plot for the Van der Pol oscillator ($a=5$) using `rk4`, showing the limit cycle.

5 Problem 3: Adaptive RK4 Integrator

5.1 Numerical Approach and Implementation

The adaptive integrator `rk4ad.m` was implemented with the header function `[tout, yout] = rk4ad(fcn, tspan, reltol, y0)`. The core numerical approach is to integrate the solution between consecutive points in the `tspan` vector. To get from one output time t_i to the next t_{i+1} , the function executes an inner `while` loop that takes adaptive micro-steps.

A primary requirement for the integrator is to produce solutions at the exact time points specified in the `tspan` vector. To achieve this, a mechanism is implemented at the start of each micro-step. If a proposed step Δt would overshoot the next target output time, t_{target} , the step size is preemptively shortened to $\Delta t = t_{\text{target}} - t_{\text{current}}$. This ensures the integration lands precisely on the target grid point. This output-matching logic is given priority over all other step-size controls, including the minimum step size floor.

The logic for each micro-step implements a standard relative error control:

1. A step size Δt is proposed. The function first checks that this step will not overshoot the target output time, t_{i+1} , and shortens it if necessary.
2. Two solutions are computed: y_C (one “coarse” step of size Δt) and y_F (two “fine” steps of size $\Delta t/2$) (as talked about in Section 2.2).
3. The absolute local error is estimated: $\text{err}_{\text{est}} = \frac{16}{15} \|y_C - y_F\|$.
4. A dynamic, relative error tolerance, tol , is calculated. This tolerance is scaled by the magnitude of the solution (using the more accurate y_F) to make the error check relative. A small absolute tolerance floor (`abstol`) is included to prevent the tolerance

from becoming zero if the solution y_F passes through zero:

$$\text{tol} = \text{reltol} \times \|y_F\| + \text{abstol}$$

5. **If (Accept) $\text{err}_{\text{est}} \leq \text{tol}$:** The step is successful. The current time t is advanced by Δt , the solution y is updated to y_F , and a new, larger Δt is calculated.
6. **If (Reject) $\text{err}_{\text{est}} > \text{tol}$:** The step has failed. The time t and solution y are not changed. A new, smaller Δt is calculated and the step is retried.

In both cases, the new step size is calculated based on the $O(\Delta t^5)$ error relationship, $\Delta t_{\text{new}} \propto (\text{tol}/\text{err}_{\text{est}})^{1/5}$ (again as shown in Section 2.2). This process, which includes step-size growth/shrink limits and a Δt floor of 10^{-4} , repeats until $t = t_{i+1}$.

This adaptive function was then tested on the same SHO and VDP systems as described in Sections 4.2.1 and 4.2.2. In the SHO test (`trk4ad_sho.m`), the integrator was run using four different relative tolerances (10^{-5} , 10^{-7} , 10^{-9} , and 10^{-11}) to verify that the resulting global error respects the requested bounds.

In the VDP test (`trk4ad_vdp.m`), a single relative tolerance of 10^{-10} was used to demonstrate the integrator's ability to efficiently solve a non-linear system while maintaining accuracy.

5.2 Results

5.2.1 SHO

The script `trk4ad_sho.m` solved the SHO system with four different relative tolerances: 10^{-5} , 10^{-7} , 10^{-9} , and 10^{-11} . The resulting absolute errors in $x(t)$ for each run are plotted in two separate figures.

Figure 5 shows the errors on a standard linear axis. This plot clearly visualizes the error for the larger tolerances (10^{-5} and 10^{-7}). However, the errors for the 10^{-9} and 10^{-11} tolerances are several orders of magnitude smaller and are not visible since they are against the x-axis, appearing to be zero.

To properly visualize the results for all four tolerances, a plot with a logarithmic y-axis is necessary, as shown in Figure 6. This log-scale plot clearly demonstrates the effectiveness of the adaptive controller. All four error curves are now distinct and in the correct order. As the `reltol` parameter is made stricter, the actual global error of the solution decreases accordingly. This confirms that the integrator is successfully enforcing the requested error tolerance across all tested scales.

5.2.2 VDP

The VDP system was solved using `reltol =  $10^{-10}$` . The results (Figures 7 and 8) are qualitatively identical to those from the fixed-step integrator, correctly capturing the limit cycle. The key advantage of the adaptive method is that it achieves this solution with a guaranteed error bound, while also being more efficient. It automatically takes small steps during the initial rapid transient and larger, more efficient steps once the solution is on the smooth limit cycle.

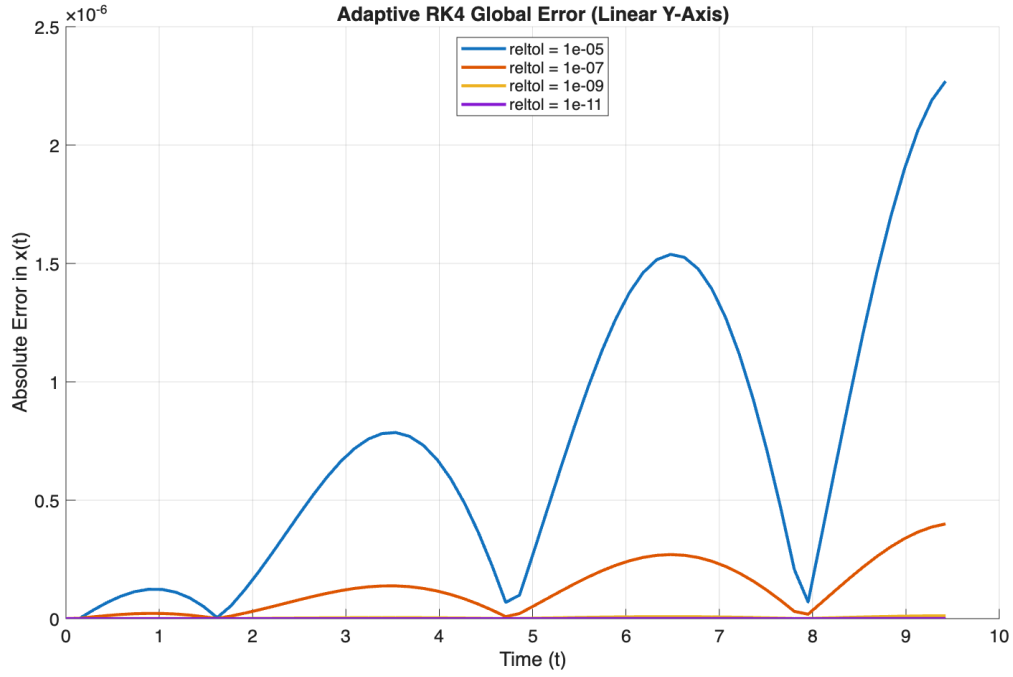


Figure 5: Global error in $x(t)$ for the SHO solved with `rk4ad` (Linear Y-Axis). The errors for 10^{-9} and 10^{-11} are indistinguishable from zero.

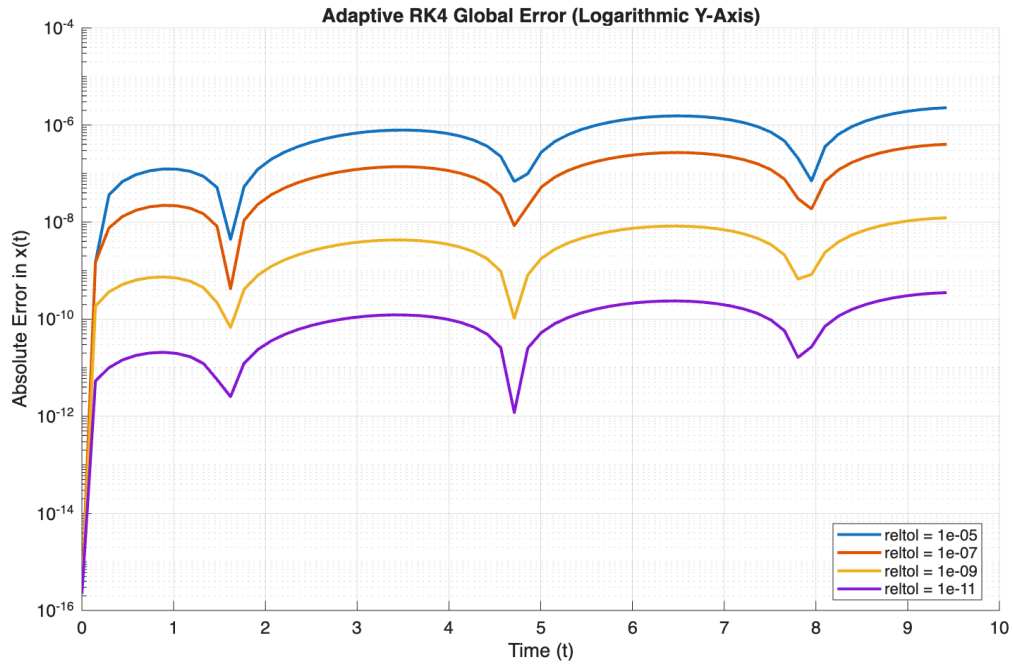


Figure 6: Global error in $x(t)$ for the SHO solved with `rk4ad` (Logarithmic Y-Axis). All four error curves are now visible and show decreasing error with stricter tolerance.

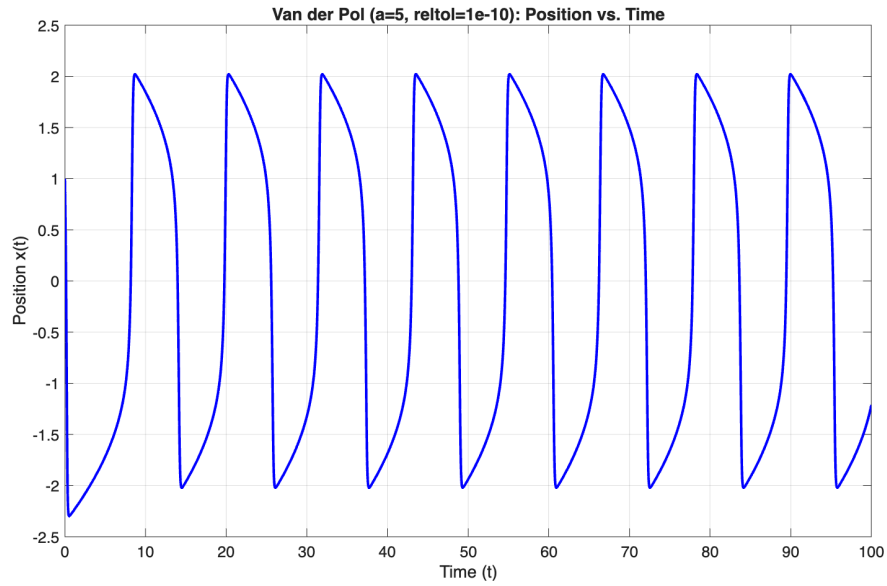


Figure 7: Position vs. Time for the Van der Pol oscillator ($a=5$) using `rk4ad` ($\text{reltol} = 1\text{e-}10$).

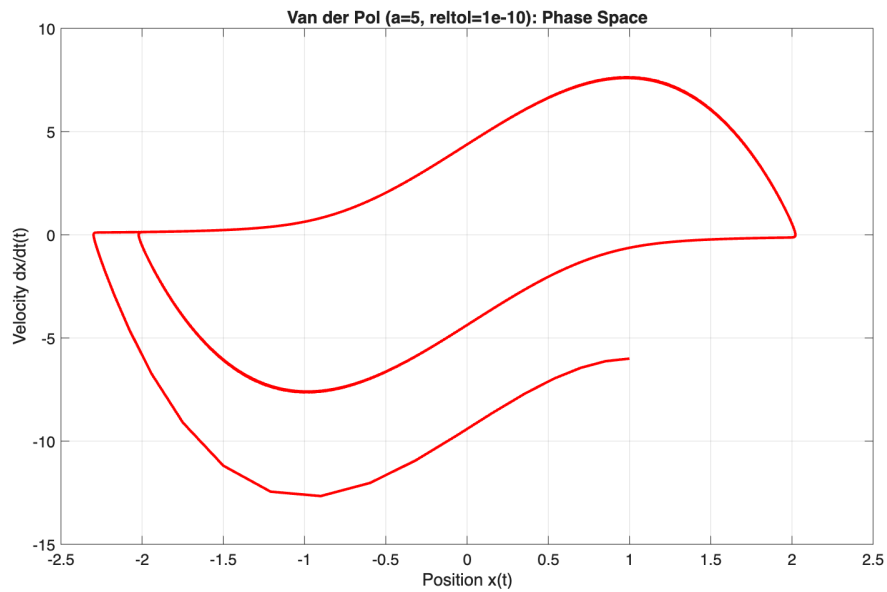


Figure 8: Phase-space plot for the Van der Pol oscillator ($a=5$) using `rk4ad`, showing the limit cycle.

6 Conclusion

This assignment successfully implemented and verified three levels of Runge-Kutta integrators: a single-step RK4 function, a fixed-step RK4 integrator, and an adaptive-step RK4 integrator. Each implementation was tested against well-understood systems, confirming their expected $O(\Delta t^5)$ local and $O(\Delta t^4)$ global error behaviors and demonstrating their capabilities in solving both linear and non-linear ODEs.

The primary challenge encountered in this assignment was ensuring the adaptive integrator correctly handled the relative tolerance input. This required multiplying the tolerance by the solution magnitude to create a dynamic error threshold, as well as implementing logic to ensure output times were matched exactly. Once these aspects were addressed, the integrator performed as expected. Another annoying issue was plotting the very small errors from the adaptive integrator, which required switching to a logarithmic scale to visualize all results effectively. When I tried to use the `semilogy` function, the plot would just plot linearly anyway, so I had to manually set the y-axis to log scale.

6.1 Use of AI

I used ChatGPT and Gemini as tools to help with specific implementation details. This included using a cell array to store outputs in the `rk4` function testers and improving data visualization, such as plotting the `trk4ad_sho` results on a logarithmic scale. I also used AI to help ensure the `rk4ad` adaptive step-size control logic was sound, including the choice of max/min factors for Δt . I made sure to verify all code suggestions, using them only as a starting point and ensuring I understood everything that was implemented.

7 Files

The following MATLAB files were created for this assignment:

- **Problem 1:** `rk4step.m`, `trk4step.m`
- **Problem 2:** `rk4.m`, `trk4_sho.m`, `trk4_vpd.m`
- **Problem 3:** `rk4ad.m`, `trk4ad_sho.m`, `trk4ad_vpd.m`